AI Evals For Engineers & PMs

Course syllabus · Instructors: Hamel Husain and Shreya Shankar

A hands-on course on evaluating and improving LLM-powered products. You build a support agent, instrument it so its behavior is measurable, find its failures with a disciplined error-analysis process, put it under a CI/CD regression suite, red-team it, and then improve both accuracy and cost.

There are nine live lectures of 60 minutes each, plus a bonus lecture on job interviewing. Additionally, there are 10+ hours of live office hours.

Module 1 — Building Agents (L1–L3)

Opens the course and creates the agent, data, scenarios, and traces that every later module uses.

L1: Building Agents, Foundations. Write a SPEC.md that pins down scope, roles, tool contracts, risk tiers, and escalation. Introduce the Three Gulfs (comprehension, specification, generalization) mapped to Analyze, Measure, and Improve. Then build a working support agent on the OpenAI Agents SDK with permissions enforced in code.

L2: Building Agents, Designing for Evaluability. Instrument the agent before it takes user traffic. Design a trace data model with nested spans, model and tool calls, permission denials, and prompt hashes. Stand up self-hosted Langfuse and ClickHouse.

L3: Building Agents, Synthetic Data & Scenarios. Build a fictional commerce world, with deterministic users, orders, and facts.yaml. Write a synthetic-data skill, generate roughly 500 support scenarios and 150 analyst scenarios, and produce a smoke report.

Module 2 — Error Analysis (L4–L5)

Turns Module 1's traces into a human-owned failure taxonomy, validated evaluators, and a prioritized failure report for Module 3.

L4: Error Analysis, Finding Failures. Why "let the agent evaluate everything" fails. Build a review loop and interface, then do open coding (read whole traces, note the first failure in plain language) and axial coding (group notes into binary failure modes and scan for saturation). Compare against published taxonomies only after building your own.

L5: Error Analysis, Measuring with Evaluators. One binary evaluator per failure mode: code checks for objective failures, LLM judges only when interpretation is needed. Judge-prompt writing, train/dev/test splits, true positive and true negative rate reporting, and freeze-and-read-test-once discipline. Estimate prevalence with bootstrap confidence intervals, and extend the same discipline to retrieval, grounding, and handoff subsystems.

Homework: label at least 60 support traces and build a taxonomy of 5–8 binary failure modes with definitions, examples, and clear boundaries.

Module 3 — CI/CD (L6)

Turns Module 2's failure report, cases, and judges into a regression suite before deployment and sampled monitoring after.
L6: CI/CD for Agents. Convert failures into test cases (input, expected result, initial state). Mechanical failures become code assertions, subjective failures become pinned judge checks. Work through cost tiers, from deterministic checks to mocked integration to full agent evals. Use pass^k for reliability versus pass@k for capability, and reset-and-replay to measure failure rate. Wire a GitHub Actions gate, then set up post-deploy monitoring: code checks on all traffic, frozen judges on a sample, and dashboards with corrected prevalence and alerts.

Module 4 — Security, Safety, and Governance (L7)

L7: Safety & Adversarial Evaluation. Map the attack surface with the OWASP Top 10 for Agentic Applications: goal hijacking, tool misuse, privilege abuse, and memory or context poisoning. Prompt injection has no reliable detector, so authorization must hold even when the model is compromised. Run a live red-team demo against the running endpoint with promptfoo, and turn successful attacks into a failing adversarial test. Add input, output, and tool guards, a human-approval flow for irreversible actions, and a governance record structured around the NIST AI Risk Management Framework, plus the legal floor set by the EU AI Act.

Module 5 — Improving Agents (L8–L9)

Uses the accumulated backlog, judges, dev and test slices, and adversarial tests to compare agent configurations on accuracy and cost, keeping multiple points on the Pareto frontier.
L8: Improving Agents: Accuracy & Joint Optimization. Build a comparable frontier (same prompt, tools, harness, workload, and suite per point), then vary one axis at a time. Route each fix to the cheapest effective layer: prompt, then tool design, then harness, then model or weights. Run a manual fix loop before automating with GEPA or a bounded improve-loop, and guard against reward hacking. Compare frontier-model baselines against an optimized variant on the held-out test slice.
L9: Improving Agents: Cost. Profile cost and find common issues: history growth, retrieval depth, and repeated tool schemas. Apply prompt caching, model routing and cascades calibrated against labeled data, and token reduction. Reach for a weights track (distillation, SFT, RL) only when prompt search is flat and failures are genuine capability limits. Finish with the upgrade drill: store frontier variants as configs, re-run the full suite whenever a model ships, redraw the frontier, and decide what to deploy, keep, or retire.
Homework (two parts): apply a prompt, tool, or harness change to your top failure mode and freeze a winner on the test slice. Then profile costs, measure a caching change, calibrate a cascade, and run the upgrade drill across at least two committed frontier configs.

Bonus — Evals Interview Prep

Common traps in eval-focused interviews and how to avoid them.

[Back to course page]